

Shahe Bannis

40286754

COMP 6381 – Final project report

1. Introduction

Dynamic point clouds are a core component to volumetric video, immersive telepresence, and autonomous sensing. Unlike traditional media, these datasets consist of dense geometric and attribute information that changes over time, which makes them extremely expensive in terms of storage and bandwidth. Therefore, there has been an effort to develop different approaches to compression like MPEG's V-PCC, G-PCC and Draco.

One of the main challenges in this space is that comparing these compression methods fairly is not straightforward. Their performance is highly sensitive to the properties contained in the input data such as scale, sampling density, noise, and temporal consistency. Two point cloud sequences that look similar can behave very differently under different compression algorithms due to these factors.

To address this, this project focuses on building a reproducible evaluation framework that applies geometry-aware preprocessing before compression.

2. System Architecture

The framework was implemented in python and packaged in Docker to ensure the reproducibility of the project. There is the preprocessing pipeline (implemented from scratch), the compression pipeline (MPEG's G-PCC compression algorithm and Google's Draco) and a web-based interface for interaction.

The preprocessing pipeline includes four main operations:

Geometric normalization, where each point cloud is centered at the origin and scaled so it fits in a fixed range. There is also a PCA based alignment that can be applied to orient the data consistently across frames.

Voxel-Grid Density Regularization, where points are grouped into voxel cells, and each occupied cell is reduced to a single representative point.

Poisson-Disk Sampling, where a greedy sampling strategy ensures that no two points are closer than a specified radius. This creates a well-spaced subset of points and avoids clustering.

MLS smoothing, where each point is locally projected onto an estimated surface using weighted neighbourhoods. This reduces noise while preserving the original structure and can be applied iteratively for stronger smoothing.

The framework provides a subprocess wrapper for external codec tools. G-PCC is MPEG's geometry-based codec that operates on octree-encoded integer coordinates and Draco is Google's codec that operates using kdtrees. The wrapper converts floating-point coordinates to integer coordinates and handles running the tool with the required settings

A flask-based interface is implemented to allow users to interact with the system. Users can upload point cloud files, adjust preprocessing settings and view the result of those operations using side-by-side 3D viewers.

3. Challenges

Preprocessing performance was a big challenge throughout the project. Without normalizing the point cloud first, the MLS smoothing operator takes a long time to complete as the code loops over every point and there is no optimization put in place due to the constraint of time. This means that it would go over 100k points at times. To counteract this, I used kdtrees to group up points together during MLS smoothing which made the preprocessing run a lot faster.

Another issue had to do with the installation of the codecs for encoding and collecting relevant metrics. Due to the number of dependencies required, they were very fragile during installation and would break often for no apparent reason. Draco was the best and most stable of them all as it was a lot laxer in terms of its dependencies and their versions.